

AURIX

Microcontroller Infineon Software Architecture

User's Manual

MCAL Library

Release v2.7

2016-08

Edition 2016-08

**Published by
Infineon Technologies AG
81726 Munich, Germany**

**© 2016 Infineon Technologies AG
All Rights Reserved.**

LEGAL DISCLAIMER

THE INFORMATION GIVEN IN THIS DOCUMENT IS GIVEN AS A HINT FOR THE IMPLEMENTATION OF THE INFINEON TECHNOLOGIES COMPONENT ONLY AND SHALL NOT BE REGARDED AS ANY DESCRIPTION OR WARRANTY OF A CERTAIN FUNCTIONALITY, CONDITION OR QUALITY OF THE INFINEON TECHNOLOGIES COMPONENT. THE RECIPIENT OF THIS DOCUMENT MUST VERIFY ANY FUNCTION DESCRIBED HEREIN IN THE REAL APPLICATION. INFINEON TECHNOLOGIES HEREBY DISCLAIMS ANY AND ALL WARRANTIES AND LIABILITIES OF ANY KIND (INCLUDING WITHOUT LIMITATION WARRANTIES OF NON-INFRINGEMENT OF INTELLECTUAL PROPERTY RIGHTS OF ANY THIRD PARTY) WITH RESPECT TO ANY AND ALL INFORMATION GIVEN IN THIS DOCUMENT.

Information

For further information on technology, delivery terms and conditions and prices, please contact the nearest Infineon Technologies Office (www.infineon.com).

Warnings

Due to technical requirements, components may contain dangerous substances. For information on the types in question, please contact the nearest Infineon Technologies Office.

Infineon Technologies components may be used in life-support devices or systems only with the express written approval of Infineon Technologies, if a failure of such components can reasonably be expected to cause the failure of that life-support device or system or to affect the safety or effectiveness of that device or system. Life support devices or systems are intended to be implanted in the human body or to support and/or maintain and sustain and/or protect human life. If they fail, it is reasonable to assume that the health of the user or other persons may be endangered.

CONFIDENTIAL

Date	Version	Change Description
2016-08-09	2.7	Updated based on review comments REV_010625.
2016-08-09	2.6	Updated based on JIRA 0000051108-327.
2016-08-04	2.5	Updated traceability tags in section 4.2
2016-07-21	2.4	Updated the added AoU tag in section 4.2
2016-05-26	2.3	REV_009703: Incorporated review comments. Added AoU tag in section 4.2. Added description of Mcal_DmaDisableChIntrTrigger API. Removed unused API information and updated section 4.3.
2015-11-02	2.2	Modified Section - 4.2 (All subsections) for I/O privileges. Added Section - 4.2.50 Mcal_UpdateRegSV and 4.2.51 Mcal_SVModeExecution for User Mode implementation. Added sequence diagrams for explanation. Review comments are fixed
2015-09-10	2.1	Modified Section - 4.2.30 Mcal_ResetSafetyENDINIT_Timed and 4.2.31 Mcal_SetSafetyENDINIT_Timed as per JIRA No. 0000051124-164
2015-06-04	2.0	Modified section 5 Access Permissions for variables as per JIRA 0000051261-79
2015-04-30	1.9	Updated section 5 Access Permissions for variables
2015-03-30	1.8	Added new section 5 Access Permissions as per JIRA 0000051100-57
2015-03-17	1.7	Added API Mcal_DmaCfgNoOfMovesPerTransfer for JIRA 0000051100-52
2015-02-20	1.6	Updated description of DMA library API's as per JIRA 0000051261-4
2014-12-09	1.5	Updated the review comments. REV_006622. Updated the file structure Updated the I/O mode for all the APIs
2014-12-05	1.4	Added description for Mcal_SafeErrorHandler, Mcal_LockResource, Mcal_UnlockResource .Removed description for Mcal_SetSafetyENDINIT and Mcal_ResetSafetyENDINIT APIs.
2014-07-17	1.3	Description updated for Mcal_ResetSafetyENDINIT_Timed API
2014-05-30	1.2	Mcal_DmaCfgNoOfMovesPerTransfer Api removed from the library
2013-08-14	1.1	Mcal_DmaInit API removed from the library
2013-06-24	1.0	Updated the file structure and API descriptions.
2012-03-01	0.1	Initial Draft version

We Listen to Your Comments

Is there any information in this document that you feel is wrong, unclear or missing?
Your feedback will help us to continuously improve the quality of this document.
Please send your proposal (including a reference to this document) to:

mcdocu.comments@infineon.com



Table of Contents	Page
1 Introduction	9
1.1 Scope	9
1.2 Abbreviations.....	9
1.3 Reference	9
2 MCALLIB Driver Overview.....	11
2.1 Software Hardware Mapping.....	11
2.1.1 DMA	11
2.1.2 SCU	12
2.2 Software Driver Description	12
3 File Structure	14
4 General description	16
5 API Documentation	17
5.1 API Type Definitions.....	17
5.1.1 Type Definition Dma_StatusType	17
5.1.2 Type Definition Dma_ChannelType	17
5.1.3 Type Definition Dma_ErrorStatusType	17
5.2 API Functions	18
5.2.1 Mcal_DmaSetSourceAddr.....	18
5.2.2 Mcal_DmaSetDestAddr.....	18
5.2.3 Mcal_DmaSetTxCount	18
5.2.4 Mcal_DmaEnableHwTransfer	19
5.2.5 Mcal_DmaDisableHwTransfer	19
5.2.6 Mcal_DmaStartTransaction.....	19
5.2.7 Mcal_DmaGetTransReqLostFlag.....	19
5.2.8 Mcal_DmaGetMoveEngErrFlags	20
5.2.9 Mcal_DmaClrErrFlags.....	20
5.2.10 Mcal_DmaResetDmaChannel.....	20
5.2.11 Mcal_DmaSetDataWidth.....	21
5.2.12 Mcal_DmaGetTransferIntStatus.....	21
5.2.13 Mcal_DmaGetIntStatus	21
5.2.14 Mcal_DmaChClrIntrFlags.....	22
5.2.15 Mcal_DmaDisableChIntrTrigger	22
5.2.16 Mcal_DmaEnableChIntrTrigger.....	22
5.2.17 Mcal_DmaCfgIntControl.....	23
5.2.18 Mcal_DmaChClrIntctEtrlFlags.....	23
5.2.19 Mcal_DmaChClrCtlCfg.....	24
5.2.20 Mcal_DmaClrAdcr	24
5.2.21 Mcal_DmaSetAdicr	24
5.2.22 Mcal_DmaEnTransReqLostIntr.....	24
5.2.23 Mcal_DmaEnableIntr.....	25
5.2.24 Mcal_DmaDisableIntr	25
5.2.25 Mcal_DmaCfgMeErrIntr	25
5.2.26 Mcal_DmaGetTransferCount	26
5.2.27 Mcal_DmaClrIntrTriggerFlag.....	26
5.2.28 Mcal_DmaCfgNoOfMovesPerTransfer	26
5.2.29 Mcal_SetENDINIT	27
5.2.30 Mcal_ResetENDINIT	27
5.2.31 Mcal_ResetSafetyENDINIT_Timed	27
5.2.32 Mcal_SetSafetyENDINIT_Timed.....	28
5.2.33 Mcal_SafetyModifyAccess	28
5.2.34 Mcal_CpuWdtModifyAccess	28
5.2.35 Mcal_SafetyCheckAccess.....	29
5.2.36 Mcal_CpuWdtCheckAccess.....	29
5.2.37 Mcal_SafetyPwSequence	29
5.2.38 Mcal_CpuWdtPwSequence	30

Table of Contents		Page
5.2.39	Mcal_SuspendAllInterrupts	30
5.2.40	Mcal_ResumeAllInterrupts	30
5.2.41	Mcal_GetCoreId	30
5.2.42	Mcal_GetDsprReMapAddress	31
5.2.43	Mcal_GetCpuAddress	31
5.2.44	Mcal_SetCpuPC	31
5.2.45	Mcal_StartCore	31
5.2.46	Mcal_GetSpinLock	32
5.2.47	Mcal_ReleaseSpinLock	32
5.2.48	Mcal_SafeErrorHandler	32
5.2.49	Mcal_LockResource	33
5.2.50	Mcal_UnlockResource	33
5.3	Access Permissions	33
6	Example Usage	34
7	Assumptions and Limitations	35

List of Figures**Page**

Figure 1	Software Hardware Mapping of Mcal Library.....	11
Figure 2	Interfaces of MCAL Lib.....	13
Figure 3	Code Structure of MCAL Library.....	14
Figure 4	Mcal_UpdateRegSV execution when IFX_MCAL_RUN_MODE_DEFINE = 0U (Supervisor Mode)	35
Figure 5	Mcal_UpdateRegSV execution when IFX_MCAL_RUN_MODE_DEFINE = 1U (User-1 Mode without OS)	35
Figure 6	Mcal_UpdateRegSV execution when IFX_MCAL_RUN_MODE_DEFINE = 2U (User-1 Mode with OS)	35

List of Tables
Page

Table 1	Variance of peripheral units/features	9
Table 2	Abbreviations used in this Document.....	9
Table 3	Code Structure	14
Table 4	Type Definition Dma_StatusType	17
Table 5	Type Definition Dma_ChannelType	17
Table 6	Type Definition Dma_ErrorStatusType	17
Table 7	Specification of API Mcal_DmaSetSourceAddr	18
Table 8	Specification of API Mcal_DmaSetDestAddr	18
Table 9	Specification of API Mcal_DmaSetTxCount.....	18
Table 10	Specification of API Mcal_DmaEnableHwTransfer.....	19
Table 11	Specification of API Mcal_DmaDisableHwTransfer	19
Table 12	Specification of API Mcal_DmaStartTransaction	19
Table 13	Specification of API Mcal_DmaGetTransReqLostFlag	19
Table 14	Specification of API Mcal_DmaGetMoveEngErrFlags	20
Table 15	Specification of API Mcal_DmaClrErrFlags	20
Table 16	Specification of API Mcal_DmaResetDmaChannel	20
Table 17	Specification of API Mcal_DmaSetDataWidth	21
Table 18	Specification of API Mcal_DmaGetTransferIntStatus	21
Table 19	Specification of API Mcal_DmaGetIntStatus.....	21
Table 20	Specification of API Mcal_DmaChClrIntrFlags	22
Table 21	Specification of API Mcal_DmaDisableChIntrTrigger	22
Table 22	Specification of API Mcal_DmaEnableChIntrTrigger	22
Table 23	Specification of API Mcal_DmaCfgIntControl	23
Table 24	Specification of API Mcal_DmaChClrIntctEtrlFlags	23
Table 25	Specification of API Mcal_DmaChClrCtlCfg	24
Table 26	Specification of API Mcal_DmaClrAdcr.....	24
Table 27	Specification of API Mcal_DmaSetAdicr	24
Table 28	Specification of API Mcal_DmaEnTransReqLostIntr	24
Table 29	Specification of API Mcal_DmaEnableIntr	25
Table 30	Specification of API Mcal_DmaDisableIntr	25
Table 31	Specification of API Mcal_DmaCfgMeErrIntr	25
Table 32	Specification of API Mcal_DmaGetTransferCount.....	26
Table 33	Specification of API Mcal_DmaClrIntrTriggerFlag	26
Table 34	Specification of API Mcal_DmaCfgNoOfMovesPerTransfer	26
Table 35	Specification of API Mcal_SetENDINIT	27
Table 36	Specification of API Mcal_ResetENDINIT	27
Table 37	Specification of API Mcal_ResetSafetyENDINIT_Timed	27
Table 38	Specification of API Mcal_SetSafetyENDINIT_Timed	28
Table 39	Specification of API Mcal_SafetyModifyAccess.....	28
Table 40	Specification of API Mcal_CpuWdtModifyAccess	28
Table 41	Specification of API Mcal_SafetyCheckAccess	29
Table 42	Specification of API Mcal_CpuWdtCheckAccess	29
Table 43	Specification of API Mcal_SafetyPwSequence	29
Table 44	Specification of API Mcal_CpuWdtPwSequence	30
Table 45	Specification of API Mcal_SuspendAllInterrupts.....	30
Table 46	Specification of API Mcal_ResumeAllInterrupts.....	30
Table 47	Specification of API Mcal_GetCoreId	30
Table 48	Specification of API Mcal_GetDsprReMapAddress	31
Table 49	Specification of API Mcal_GetCpuAddress.....	31
Table 50	Specification of API Mcal_SetCpuPC	31
Table 51	Specification of API Mcal_StartCore	31
Table 52	Specification of API Mcal_GetSpinLock.....	32
Table 53	Specification of API Mcal_ReleaseSpinLock	32
Table 54	Specification of API Mcal_SafeErrorHandler	32
Table 55	Specification of API Mcal_LockResource	33
Table 56	Specification of API Mcal_UnlockResource	33

List of Code Listings

Page

Code Listing 1	Example – MCAL LIB usage in ADC driver.....	34
Code Listing 2	Example – MCAL LIB usage in SPI driver	35

1 Introduction

This User Manual provides information regarding the functionality, and API implementation for the MCAL Library.

1.1 Scope

The MCAL Library provides various APIs to configure the Dma, Wdg ENDINIT functionalities as well as to get CPU core id. These APIs are used by multiple driver modules and hence are placed as part of MCAL Library

Table 1 Variance of peripheral units/features

Microcontroller	Number of DMA channels
TC23x/TC22x	16
TC26X	48
TC27X	64
TC29X	128

1.2 Abbreviations

The abbreviations used inside this document are explained in [Table 2](#).

Table 2 Abbreviations used in this Document

Abbreviations	Explanation
μC	Microcontroller
API	Application Programming Interface
AUTOSAR	AUTomotive Open System ARchitecture
DMA	Direct Memory Access
ID	Identifier
ISR	Interrupt Service Routine
MCAL	MicroController Abstraction Layer
SW	Software

1.3 Reference

This section contains references to all documents relevant to this Architecture Specification.

- [1] Refer to the latest version for the documents without version information
- [2] TC27x_MC-ISAR_SRD.xls, Aurix_Gen1_PRD_v2.0.pdf Product Requirement Document
- [3] TC27x Target Specification : tc27x_um_v1.4.pdf
- [4] TC27xC Target Specification : tc27xC_um_v2.2.pdf
- [5] TC26x Target Specification : tc26xB_um_v1.3.pdf
- [6] TC29x Target Specification : tc29xB_um_v1.3.pdf
- [7] TC23x/TC22x Target Specification : tc23x_tc22x_um_v1.1.pdf

[8] TC27xD Target Specification : tc27xD_um_v2.2.pdf

2 MCALLIB Driver Overview

2.1 Software Hardware Mapping

This section introduces the user to the HW features used for implementation. The [Figure 1](#) illustrates the peripherals and their interaction with the Library.

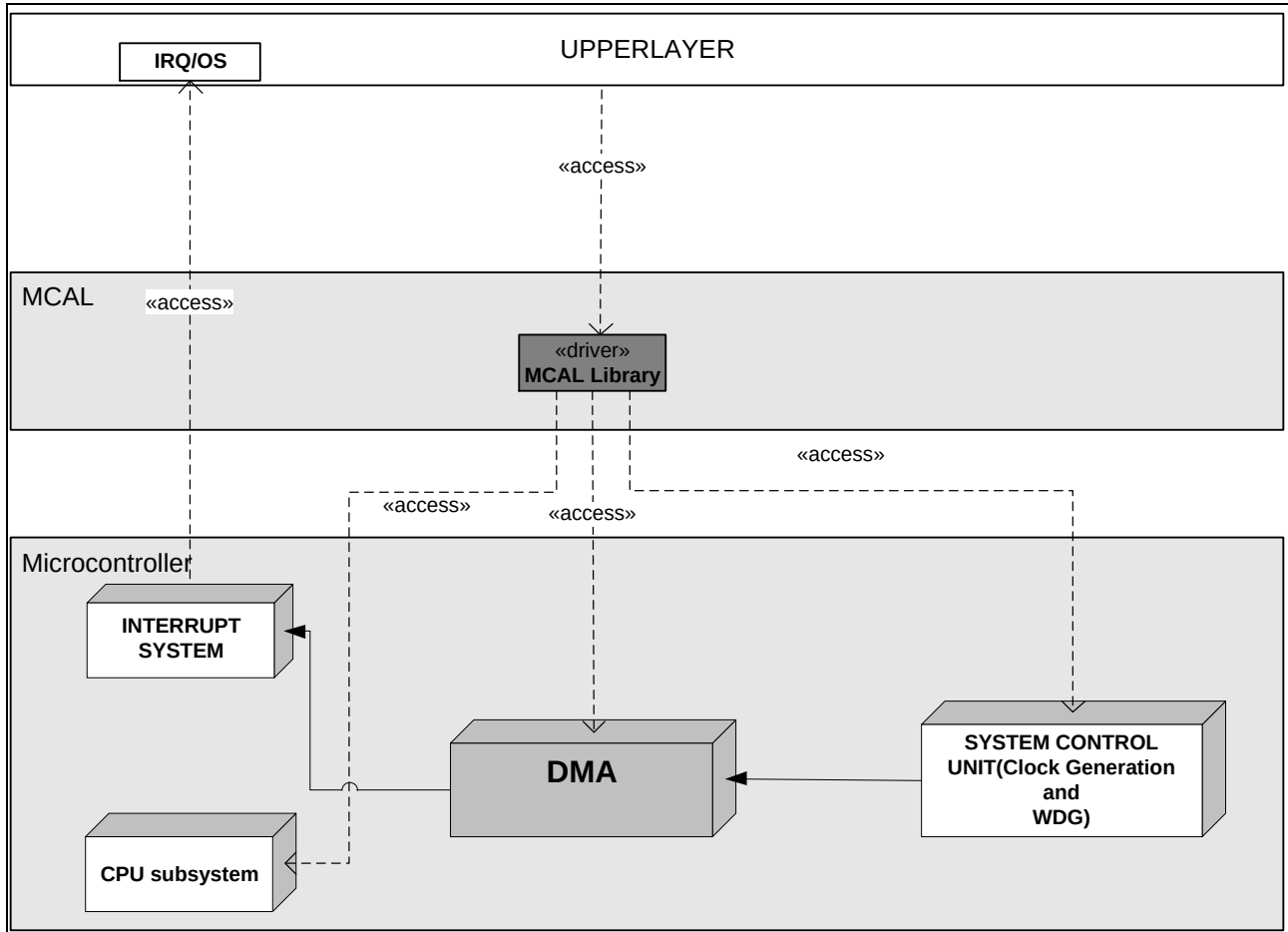


Figure 1 Software Hardware Mapping of Mcal Library.

The different HW modules needed by the driver and their purpose is explained below.

2.1.1 DMA

The DMA Controller transfers data from data source locations to data destination locations without intervention of the CPU or other on-chip devices. DMA moves can be requested either by hardware or by software. DMA hardware requests are triggered by specific request lines from the peripheral modules or from other DMA channels. DMA service are used by different peripherals to move data from one location to another.

MCAL Lib provides APIs to move data from source location to destination depending on the driver configuration that calls these APIs. If the driver calls DMA related APIs, it will configure DMA as hardware channel, the DMA transaction is triggered by the requesting peripherals.

2.1.2 SCU

The Endinit Functions

There are a number of registers in the Aurix that are usually programmed only once during the initialization sequence of the system or application. Modification of such registers during normal application run can have a severe impact on the overall operation of modules or the entire system. While the Supervisor Mode and the Access Protection scheme provide a certain level of protection against unintentional modifications, they may not be sufficient to protect against all unintended accesses to system-critical registers.

The Aurix provides one more level of protection for such registers via the various Endinit features. This is a highly secure write-protection scheme that makes unintentional modifications of registers protected by this feature nearly impossible. The Endinit feature consists of an ENDINIT bit incorporated in each WDT control register. Registers protected via an Endinit determine whether or not writes are enabled. Writes are only enabled if the corresponding ENDINIT = 0 AND Supervisor Mode is active. Write attempts if this condition is not true will be discarded and the register contents will not be modified in this case. The BCU controls the further operation following a discarded write access. To get the highest level of security, writes to the ENDINIT bits are protected by a highly secure access protection scheme implemented in the WDTs. This is a complex procedure that makes it nearly impossible for ENDINIT bits to be modified unintentionally. In addition, each WDT monitors ENDINIT bit modifications by starting a time-out sequence each time software opens access to the critical registers through clearing the corresponding ENDINIT bit. If the time out period ends before the corresponding ENDINIT bit is set again, a malfunction of the software is assumed and a Watchdog fault response is generated. The access-protection scheme and the Endinit time-out operation of the WDTs is described in the following sections. In the register overview tables for each module (including the SCU itself) the registers which are protected via each Endinit type are identified in the column describing write accesses as follows:

- “CE0” - writeable only when CPU0 ENDINIT bit is zero
- “CE1” - writeable only when CPU1 ENDINIT bit is zero
- “CE2” - writeable only when CPU2 ENDINIT bit is zero
- “E” - writeable when any (one or more) CPUx ENDINIT bit is zero
- “SE” - writeable only when Safety ENDINIT bit is zero
- None of the above - accessible at any time

2.2 Software Driver Description

The [Figure 2](#) shows the Application Programming Interface (API) for the DMA Driver.

The APIs of the Driver are explained in detail in chapter [\[5.2\]](#)

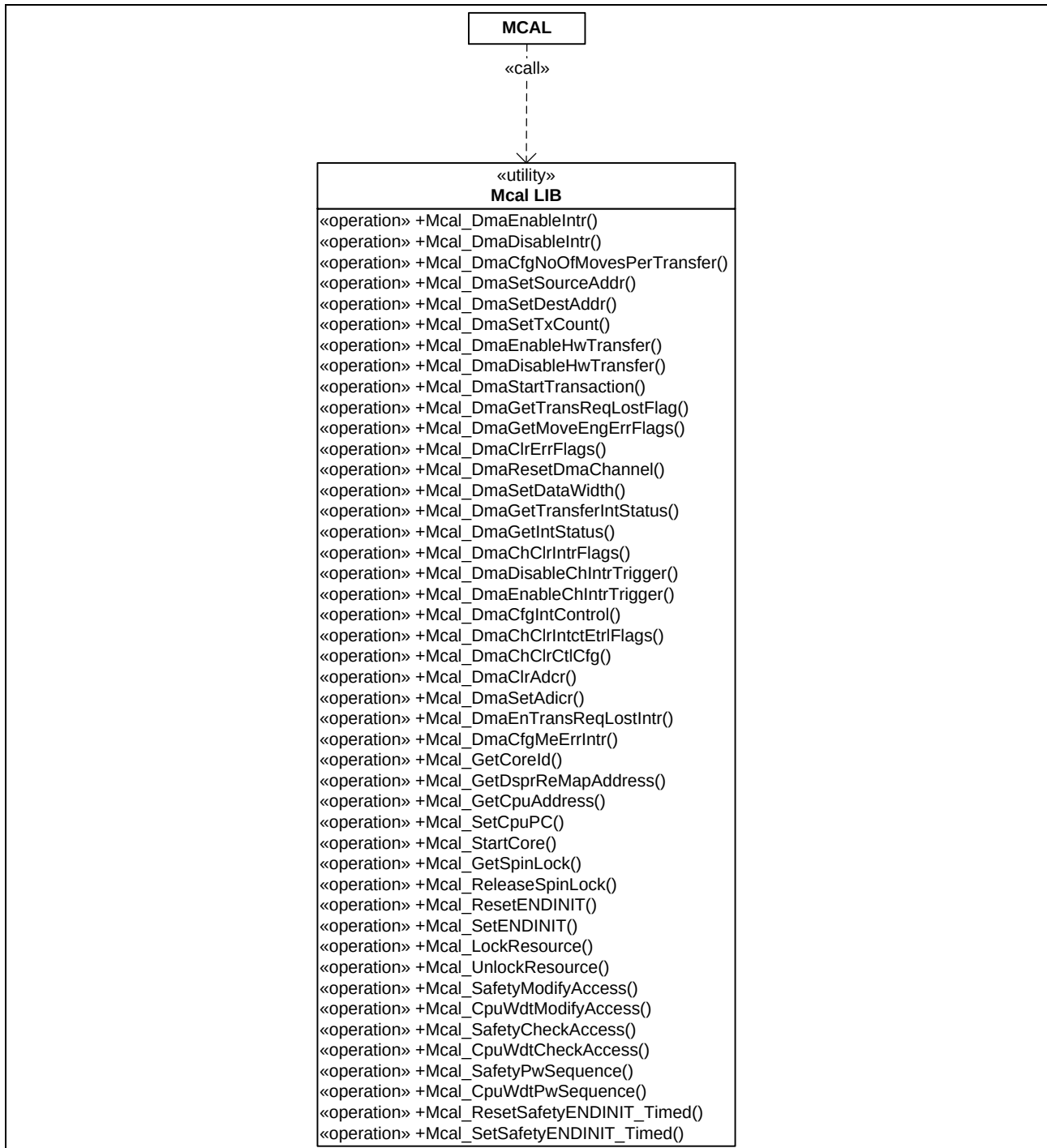


Figure 2 Interfaces of MCAL Lib

3 File Structure

This section provides details about the MCAL library files and other related files.

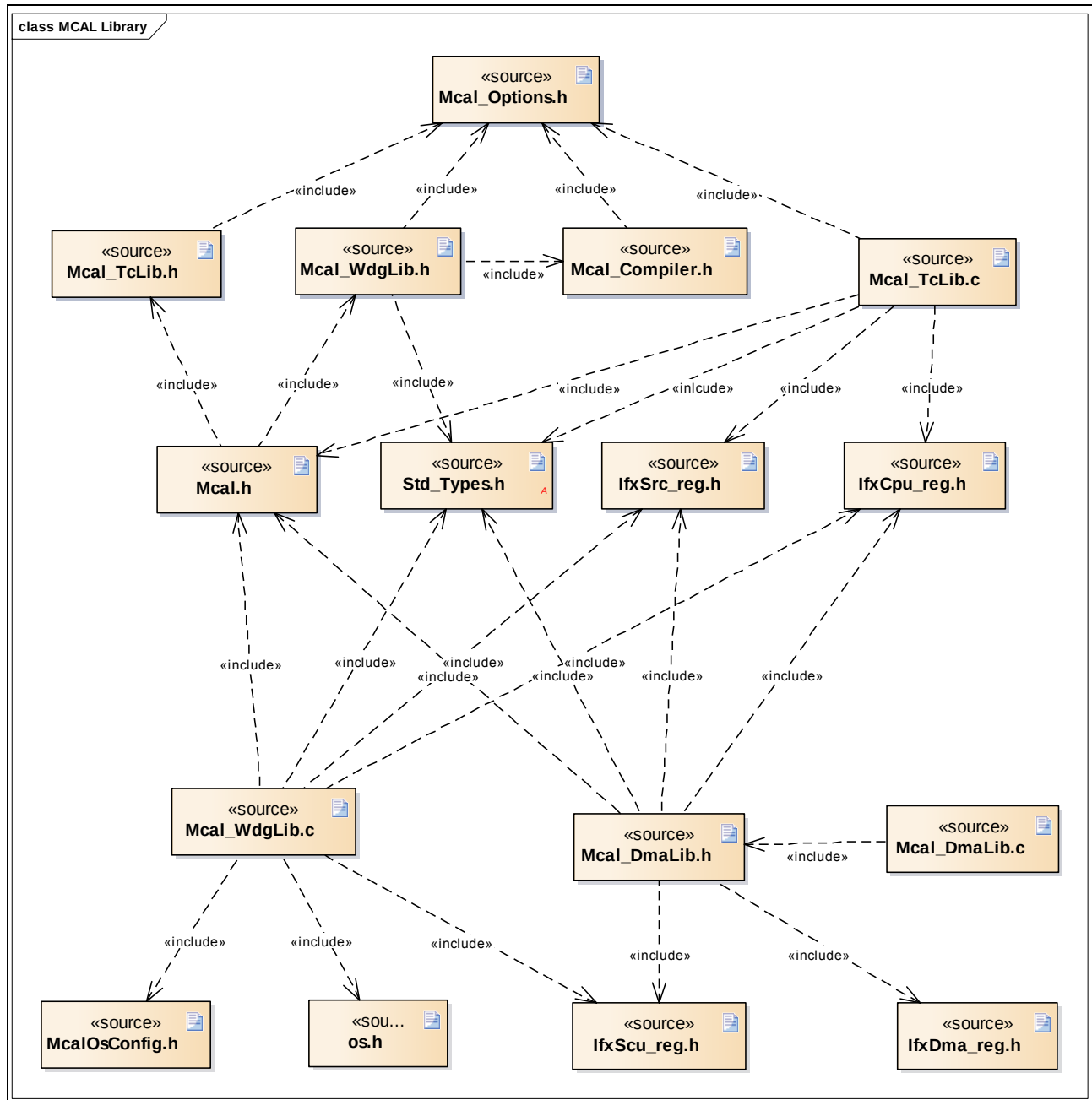


Figure 3 Code Structure of MCAL Library

Table 3 Code Structure

File Name	Description
Mcal_TcLib.h	The header file exports the Mcal Tricore library type definitions and functions
Mcal_TcLib.c	Mcal Tricore library function implementation
Mcal_WdgLib.h	The header file exports the Mcal Wdg library type definitions and

File Name	Description
	functions
Mcal_WdgLib.c	Mcal Wdg library function implementation
Mcal_DmaLib.h	The header file exports the Mcal DMA library type definitions and functions
Mcal_DmaLib.c	Mcal DMA library function implementation
IfxDma_reg.h	Includes the register definition file for DMA.
Std_Types.h	Standard data types to be used are declared here.
IfxScu_reg.h	Includes the register definition file for SCU.
IfxCpu_reg.h	Includes the register definition file for CPU.
IfxSrc_reg.h	Includes the register definition file for SRC.
Mcal.h	Provides library functions for MCAL layer.
Mcal_Compiler.h	This header file exports Mcal compiler specific functions and macros
Mcal_Options.h	This header file configures pre-compile switches stand alone Mcal/SafeTlib
McalOsConfig.h	The header file configures on required OS
Os.h	The header file exports the functionality of OS driver.

4 General description

OS_KERNAL_TYPE

OS_KERNEL_TYPE is the macro

- When it is defined MCAL utilises OS functionalities like ResumeAllInterrupts, SuspendAllInterrupts etc. by including relevant OS header files.
- When it is not defined, MCAL uses will make use of own functionalities like __enable (to enable global interrupt).

5 API Documentation

The API documentation describes the syntax and semantics of the API functions as well as it gives some additional implementation specific behavior description.

5.1 API Type Definitions

This section describes all the type definitions that are used by APIs.

5.1.1 Type Definition Dma_StatusType

Table 4 Type Definition Dma_StatusType

Syntax:	Dma_StatusType
Type:	unsigned int
File:	Mcal_DmaLib.h
Range:	0 1
Description:	This type defines a range of specific status for DMA Driver. It informs about the DMA channel status, Interrupt status and can be obtained by calling the API service such as Mcal_DmaGetTransferIntStatus.

5.1.2 Type Definition Dma_ChannelType

Table 5 Type Definition Dma_ChannelType

Syntax:	Dma_ChannelType
Type:	unsigned int
File:	Mcal_DmaLib.h
Range:	0 to 127 Dma Channel number passed onto the API
Description:	This type defines a range of Dma channel numbers, that are passed to the API

5.1.3 Type Definition Dma_ErrorStatusType

Table 6 Type Definition Dma_ErrorStatusType

Syntax:	Dma_ErrorStatusType
Type:	unsigned int
File:	Mcal_DmaLib.h
Range:	0 : indicates No Error 1 : indicates Error occurred
Description:	This type defines a specific Error status for DMA Driver. It informs about the API error status and can be obtained calling the API service such as Mcal_DmaGetTransReqLostFlag.

5.2 API Functions

This chapter describes the API interfaces provided by the MCAL Library. It is the responsibility of the user of the API to ensure that the correct values are provided as input

5.2.1 Mcal_DmaSetSourceAddr

Table 7 Specification of API Mcal_DmaSetSourceAddr

Syntax:	void Mcal_DmaSetSourceAddr (Dma_ChannelType Channel, uint32 Address)	
Parameters (in):	Channel	Channel ID.
	Address	32-bit start address
Parameters (out):	None	
Return value:	None	
Description:	This service sets the 32-bit start address of the source buffer for the required channel. Maps to DMA_SADRz register.	
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.	

5.2.2 Mcal_DmaSetDestAddr

Table 8 Specification of API Mcal_DmaSetDestAddr

Syntax:	void Mcal_DmaSetDestAddr(Dma_ChannelType Channel , uint32 Address)	
Parameters (in):	Channel	Channel ID.
	Address	32-bit destination address
Parameters (out):	None	
Return value:	None	
Description:	This service sets the 32-bit start address of the destination buffer for the required channel. Maps to DMA_DADRz register.	
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.	

5.2.3 Mcal_DmaSetTxCount

Table 9 Specification of API Mcal_DmaSetTxCount

Syntax:	void Mcal_DmaSetTxCount (Dma_ChannelType Channel, uint32 ReloadValue)	
Parameters (in):	Channel	Channel ID.
	Reload value	9-bit transfer count value Inactive DMA channel transfer reload value.
Parameters (out):	None	
Return value:	None	
Description:	This service sets the transfer reload value per Inactive DMA channel. Maps to DMA_CHCFGRz.TREL	
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.	

5.2.4 Mcal_DmaEnableHwTransfer

Table 10 Specification of API Mcal_DmaEnableHwTransfer

Syntax:	void Mcal_DmaEnableHwTransfer (Dma_ChannelType Channel)	
Parameters (in):	Channel	DMA Channel Number
Parameters (out):	None	
Return value:	None	
Description:	This service enables the hardware transfer request for the required channel. Maps to HW register DMA_TSRz.ECH	
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.	

5.2.5 Mcal_DmaDisableHwTransfer

Table 11 Specification of API Mcal_DmaDisableHwTransfer

Syntax:	void Mcal_DmaDisableHwTransfer(Dma_ChannelType Channel)	
Parameters (in):	Channel	DMA Channel Number
Parameters (out):	None	
Return value:	None	
Description:	This service disables the hardware transfer request for the required channel. Maps to HW register DMA_TSRz.DCH.	
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.	

5.2.6 Mcal_DmaStartTransaction

Table 12 Specification of API Mcal_DmaStartTransaction

Syntax:	void Mcal_DmaStartTransaction(Dma_ChannelType Channel)	
Parameters (in):	Channel	Channel ID.
Parameters (out):	None	
Return value:	None	
Description:	This service sets a transaction request for a DMA channel. Maps to DMA_CHCSRz.SCH. When setting DMA_CHCSRz.SCH, TSRz.CH becomes set to indicate that a DMA request is pending for DMA channel z.	
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.	

5.2.7 Mcal_DmaGetTransReqLostFlag

Table 13 Specification of API Mcal_DmaGetTransReqLostFlag

Syntax:	Dma_ErrorStatusType Mcal_DmaGetTransReqLostFlag (Dma_ChannelType Channel)	
Parameters (in):	Channel	Channel ID.

Parameters (out):	None	
Return value:	Dma_ErrorStatusType	Dma error status: 0 : No Errors 1 : Error occurred
Description:	This service returns the Transaction request lost status. Maps to DMA_TSRz.TRL It returns 1, if a new DMA request was detected while TSRz.CH=1 (request lost event).	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

5.2.8 Mcal_DmaGetMoveEngErrFlags

Table 14 Specification of API Mcal_DmaGetMoveEngErrFlags

Syntax:	Dma_ErrorStatusType Mcal_DmaGetMoveEngErrFlags(uint8 MoveEngine)	
Parameters (in):	MoveEngine	Move Engine number.
Parameters (out):	None	
Return value:	Dma_ErrorStatusType	Dma error status: 0 : No Errors 1 : Error/s occurred
Description:	This service returns the error status of the DMA move engine. It detects following MEx errors from DMA_ERRSRx register : SER (Source Error), DER (Destination Error), SPBER (SPB Bus Error), SRIER (SRI Bus Error), RAMER (RAM Error), SLLER (Safe Linked List Error), DLLER (DMA Linked List Error)	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

5.2.9 Mcal_DmaClrErrFlags

Table 15 Specification of API Mcal_DmaClrErrFlags

Syntax:	void Mcal_DmaClrErrFlags(Dma_ChannelType Channel)	
Parameters (in):	Channel	Channel ID.
Parameters (out):	None	
Return value:	None	
Description:	This service Clear the DMA channel transaction request lost interrupt flag TSRz.TRL. Maps to HW register DMA_TSRz.CTL	
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.	

5.2.10 Mcal_DmaResetDmaChannel

Table 16 Specification of API Mcal_DmaResetDmaChannel

Syntax:	void Mcal_DmaResetDmaChannel(Dma_ChannelType Channel)	
Parameters (in):	Channel	Channel ID.
Parameters (out):	None	
Return value:	None	

Description:	This service forces the DMA channel to stop its current transaction and reset the DMA Channel. Maps to HW register DMA_TSRz.RST.
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.

5.2.11 Mcal_DmaSetDataWidth

Table 17 Specification of API Mcal_DmaSetDataWidth

Syntax:	void Mcal_DmaSetDataWidth(Dma_ChannelType Channel, uint32 DataWidth)	
Parameters (in):	Channel	Channel ID.
	DataWidth	3-bit channel data width. Inactive DMA channel data width.
Parameters (out):	None	
Return value:	None	
Description:	Updates the Dma channel data width in Channel Control register. Maps to DMA_CHCFGRz.CHDW	
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.	

5.2.12 Mcal_DmaGetTransferIntStatus

Table 18 Specification of API Mcal_DmaGetTransferIntStatus

Syntax:	Dma_StatusType Mcal_DmaGetTransferIntStatus(Dma_ChannelType Channel)	
Parameters (in):	Channel	Channel ID.
Parameters (out):	None	
Return value:	Dma_StatusType	0: A channel interrupt has not been detected. 1: A channel interrupt has been detected.
Description:	This service returns the Inactive DMA channel interrupt trigger flag status Maps to DMA_CHCSRz.ICH.	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode	

5.2.13 Mcal_DmaGetIntStatus

Table 19 Specification of API Mcal_DmaGetIntStatus

Syntax:	Dma_StatusType Mcal_DmaGetIntStatus(Dma_ChannelType Channel)	
Parameters (in):	Channel	Channel ID.
Parameters (out):	None	
Return value:	Dma_StatusType	0: A channel interrupt has not been detected. Non Zero: A channel interrupt has been detected.
Description:	This service returns the interrupt status (IPM, ICH, WRPD & WRPS bit fields) of the	

	required channel. Maps to HW register DMA_CHCSRz.IPM, DMA_CHCSRz.ICH, DMA_CHCSRz.WRPD and DMA_CHCSRz.WRPS
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode

5.2.14 Mcal_DmaChClrIntrFlags

Table 20 Specification of API Mcal_DmaChClrIntrFlags

Syntax:	void Mcal_DmaChClrIntrFlags(Dma_ChannelType Channel)	
Parameters (in):	Channel	Channel ID.
Parameters (out):	None	
Return value:	None	
Description:	This Service clears the following interrupts: Clear Wrap Buffer Interrupt: API clears of both the DMA channel source and destination wrap buffer interrupts stored in DMARAM at locations CHSRz.WRPS and CHSRz.WRPD by setting DMA_CHCSRz.CWRP Clear Interrupt for DMA Channel: API clears of the DMA channel interrupt flags stored in DMARAM locations CHSRz.ICH and CHSRz.IPM by setting DMA_CHCSRz.CICH Clear Transaction Request Lost for DMA Channel: API clears of the DMA channel transaction request lost interrupt flag TSRz.TRL.	
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.	

5.2.15 Mcal_DmaDisableChIntrTrigger

Table 21 Specification of API Mcal_DmaDisableChIntrTrigger

Syntax:	void Mcal_DmaDisableChIntrTrigger(Dma_ChannelType Channel)	
Parameters (in):	Channel	Channel ID.
Parameters (out):	None	
Return value:	None	
Description:	This service disables the channel interrupt trigger. Maps to DMA_CHCSRz.SIT	
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.	

5.2.16 Mcal_DmaEnableChIntrTrigger

Table 22 Specification of API Mcal_DmaEnableChIntrTrigger

Syntax:	void Mcal_DmaEnableChIntrTrigger(Dma_ChannelType Channel)	
Parameters (in):	Channel	Channel ID.
Parameters (out):	None	

Return value:	None
Description:	This service enables the channel interrupt trigger. Maps to DMA_CHCSRz.SIT
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.

5.2.17 Mcal_DmaCfgIntControl

Table 23 Specification of API Mcal_DmaCfgIntControl

Syntax:	void Mcal_DmaCfgIntControl(Dma_ChannelType Channel, uint32 value)	
Parameters (in):	Channel	Channel ID.
	Value	2-bit value interrupt control value: 00B: No interrupt trigger will be generated on changing the TCOUNT value. The bit CHSRz.ICH is set when TCOUNT equals IRDV. 01B: No interrupt trigger will be generated on changing the TCOUNT value. The bit CHSRz.ICH is set when TCOUNT is decremented 10B: Interrupt trigger is generated and bit CHSRz.ICH is set on changing the TCOUNT value and TCOUNT equals IRDV 11B: Interrupt trigger is generated and bit CHSRz.ICH is set each time TCOUNT is decremented
Parameters (out):	None	
Return value:	None	
Description:	This service configures Channel Interrupt Control with the given value. Maps to DMA_ADICRz.INTCT.	
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.	

5.2.18 Mcal_DmaChClrIntctEtrlFlags

Table 24 Specification of API Mcal_DmaChClrIntctEtrlFlags

Syntax:	void Mcal_DmaChClrIntctEtrlFlags (Dma_ChannelType Channel)	
Parameters (in):	Channel	Channel ID.
Parameters (out):	None	
Return value:	None	
Description:	This service clears the Interrupt Control Register bits mapping to DMA_ADICRz.INTCT and DMA_ADICRz.ETRL	
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.	

5.2.19 Mcal_DmaChClrCtlCfg

Table 25 Specification of API Mcal_DmaChClrCtlCfg

Syntax:	void Mcal_DmaChClrCtlCfg (Dma_ChannelType Channel)	
Parameters (in):	Channel	Channel ID.
Parameters (out):	None	
Return value:	None	
Description:	This service clears the DMA Channel Control Register. Clears DMA_CHCFGRz register. Clears DMA_TSRz.ECH and DMA_TSRz.DCH register..	
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.	

5.2.20 Mcal_DmaClrAdcr

Table 26 Specification of API Mcal_DmaClrAdcr

Syntax:	void Mcal_DmaClrAdcr(Dma_ChannelType Channel)	
Parameters (in):	Channel	Channel ID.
Parameters (out):	None	
Return value:	None	
Description:	This service clears the lower 20-bits of Address control register. Maps to DMA_ADICRz	
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.	

5.2.21 Mcal_DmaSetAdicr

Table 27 Specification of API Mcal_DmaSetAdicr

Syntax:	void Mcal_DmaSetAdicr(Dma_ChannelType Channel, uint32 value)	
Parameters (in):	Channel	Channel ID.
	Value	Address and Interrupt control register value
Parameters (out):	None	
Return value:	None	
Description:	This service updates the Address control register with the passed value. Maps to DMA_ADICRz	
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.	

5.2.22 Mcal_DmaEnTransReqLostIntr

Table 28 Specification of API Mcal_DmaEnTransReqLostIntr

Syntax:	void Mcal_DmaEnTransReqLostIntr(Dma_ChannelType Channel)	
----------------	--	--

Parameters (in):	Channel	Channel ID.
Parameters (out):	None	
Return value:	None	
Description:	This service enables Transaction Request Lost Interrupt. Maps to DMA_ADICRz.ETRL	
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.	

5.2.23 Mcal_DmaEnableIntr

Table 29 Specification of API Mcal_DmaEnableIntr

Syntax:	void Mcal_DmaEnableIntr(Dma_ChannelType Channel)	
Parameters (in):	Channel	Channel ID.
Parameters (out):	None	
Return value:	None	
Description:	Function to Enable Dma Hw channel Interrupt. Maps to SRC_DMACHz	
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.	

5.2.24 Mcal_DmaDisableIntr

Table 30 Specification of API Mcal_DmaDisableIntr

Syntax:	void Mcal_DmaDisableIntr(Dma_ChannelType Channel)	
Parameters (in):	Channel	Channel ID.
Parameters (out):	None	
Return value:	None	
Description:	Function to Disable the Dma Hw channel Interrupt. Maps to SRC_DMACHz	
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.	

5.2.25 Mcal_DmaCfgMeErrIntr

Table 31 Specification of API Mcal_DmaCfgMeErrIntr

Syntax:	void Mcal_DmaCfgMeErrIntr (uint8 Enable)	
	Enable	1 – Enable the Move Engine Error Interrupt. 0 – No action
Parameters (out):	None	
Return value:	None	
Description:	Function to enable/disable the move engine error interrupt.	

	Maps to DMA_ERRINTR.SIT
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.

5.2.26 Mcal_DmaGetTransferCount

Table 32 Specification of API Mcal_DmaGetTransferCount

Syntax:	void Mcal_DmaGetTransferCount (Dma_ChannelType Channel)	
	Channel	Channel ID.
Parameters (out):	None	
Return value:	None	
Description:	This service gets the DMA transfer count for an inactive DMA channel. Maps to DMA_CHCSRz.TCOUNT	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

5.2.27 Mcal_DmaClrIntrTriggerFlag

Table 33 Specification of API Mcal_DmaClrIntrTriggerFlag

Syntax:	void Mcal_DmaClrIntrTriggerFlag (Dma_ChannelType Channel)	
Parameters (in):	Channel	Channel ID.
Parameters (out):	None	
Return value:	None	
Description:	Function to Disable the interrupt of the required DMA channel. Software clear of the DMA channel interrupt flags stored in DMARAM locations CHSRz.ICH and CHSRz.IPM. Maps to DMA_CHCSRz.CICH	
I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.	

5.2.28 Mcal_DmaCfgNoOfMovesPerTransfer

Table 34 Specification of API Mcal_DmaCfgNoOfMovesPerTransfer

Syntax:	void Mcal_DmaCfgNoOfMovesPerTransfer (Dma_ChannelType Channel, uint8 NoOfMoves)	
Parameters (in):	Channel	Channel ID.
	NoOfMoves	No. of Moves to be configured. Valid values: 1,2,3,5,4,8,9,16
Parameters (out):	None	
Return value:	None	
Description:	Function to configure the number of moves for the DMA channel. Maps to DMA_CHCFGRz.BLKM	

I/O Mode:	This API shall be invoked when the CPU is in Supervisor Mode. It can be also invoked when the CPU is in User-1 Mode, provided pre-compile macro IFX_MCAL_RUN_MODE_DEFINE = 1U or 2U.
------------------	--

5.2.29 Mcal_SetENDINIT

Table 35 Specification of API Mcal_SetENDINIT

Syntax:	void Mcal_SetENDINIT(void)	
Parameters (in):	None	
Parameters (out):	None	
Return value:	None	
Description:	Sets the ENDINIT bit in the WDT_CON0 register in order to enable the write-protection for registers protected via the EndInit feature (ie. WDT_CON1, BTV, BIV, ISP, mod_CLC). On error Mcal_SafeErrorHandler is called. Please refer description of Mcal_SafeErrorHandler for details.	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

5.2.30 Mcal_ResetENDINIT

Table 36 Specification of API Mcal_ResetENDINIT

Syntax:	void Mcal_ResetENDINIT(void)	
Parameters (in):	None	
Parameters (out):	None	
Return value:	None	
Description:	Clears the ENDINIT bit in the WDT_CON0 register in order to disable the write-protection for registers protected via the EndInit feature (ie. WDT_CON1, BTV, BIV, ISP, mod_CLC). On error Mcal_SafeErrorHandler is called. Please refer description of Mcal_SafeErrorHandler for details.	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

5.2.31 Mcal_ResetSafetyENDINIT_Timed

Table 37 Specification of API Mcal_ResetSafetyENDINIT_Timed

Syntax:	void Mcal_ResetSafetyENDINIT_Timed (uint32 TimeOut)	
Parameters (in):	TimeOut	Count of no of times to try for SafetyEndInit
Parameters (out):	None	
Return value:	None	
Description:	This function resets the safety watch dog endinit protection. There is a resource protection available (as multiple cores can avail the resource) for the safety watch dog. Initially the function tries to acquire the spin lock by looping "timeout" times, in case of a failure (i.e. spin lock not available within the allotted time) Mcal_SafeErrorHandler function is called. This function is called only from MCAL or SafeTlib. Generally sufficiently large timeout value is given, so that the resource is ensured to be available. From MCAL perspective, this function is needed only in Initialization routine that too only within MCU and PORT. Note: This API should not be called from Interrupt Context	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

5.2.32 Mcal_SetSafetyENDINIT_Timed

Table 38 Specification of API Mcal_SetSafetyENDINIT_Timed

Syntax:	void Mcal_SetSafetyENDINIT_Timed (void)	
Parameters (in):	None	
Parameters (out):	None	
Return value:	None	
Description:	This function sets the safety watch dog endinit protection and releases the acquired spinlock. In case of a failure Mcal_SafeErrorHandler function is called. This function is called only from MCAL or SafeTlib. From MCAL perspective, this function is needed only in Initialization routine that too only within MCU and PORT. Note: This API should not be called from Interrupt Context	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

5.2.33 Mcal_SafetyModifyAccess

Table 39 Specification of API Mcal_SafetyModifyAccess

Syntax:	void Mcal_SafetyModifyAccess (const uint32 NewPassword, const uint32 NewReload)	
Parameters (in):	NewPassword	New password to be set if value > MCAL_WDT_PASSWORD_MAX this value is ignored and the old password will be restored.
	NewReload	New timer reload value to be used if value > MCAL_WDT_RELOAD_MAX this value will be ignored and the old timer value will be restored
Parameters (out):	None	
Return value:	None	
Description:	Sets the new values for Password and/or timer reload value available for global safety watchdogs Proper password is calculated, new values restored on modify access. Disable interrupts during unlocked phase.	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

5.2.34 Mcal_CpuWdtModifyAccess

Table 40 Specification of API Mcal_CpuWdtModifyAccess

Syntax:	void Mcal_CpuWdtModifyAccess (const uint32 NewPassword, const uint32 NewReload)	
Parameters (in):	NewPassword	New password to be set if value > MCAL_WDT_PASSWORD_MAX this value is ignored and the old password will be restored.
	NewReload	New timer reload value to be used if value > MCAL_WDT_RELOAD_MAX this value will be ignored and the old timer value will be restored
Parameters (out):	None	
Return value:	None	

Description:	Sets the new values for Password and/or timer reload value available for CPU0, CPU1 and CPU2 watchdogs. Proper password is calculated and new values restored on modify access. Disable interrupts during unlocked phase.
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.

5.2.35 Mcal_SafetyCheckAccess

Table 41 Specification of API Mcal_SafetyCheckAccess

Syntax:	void Mcal_SafetyCheckAccess (const uint32 CheckPassword, const uint32 CheckTimer)	
Parameters (in):	CheckPassword	Password to be used for check access this value will be checked always and must match the required password
	CheckTimer	Timer value to be checked if value > MCAL_WDT_RELOAD_MAX this value will be not be used for timer check. The old timer value will be used for check access.
Parameters (out):	None	
Return value:	None	
Description:	Executes check access to verify password/timer (optional) available for global safety watchdogs Given password and timer are used on check access	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

5.2.36 Mcal_CpuWdtCheckAccess

Table 42 Specification of API Mcal_CpuWdtCheckAccess

Syntax:	void Mcal_CpuWdtCheckAccess (const uint32 CheckPassword, const uint32 CheckTimer)	
Parameters (in):	CheckPassword	Password to be used for check access this value will be checked always and must match the required password
	CheckTimer	Timer value to be checked if value > MCAL_WDT_RELOAD_MAX this value will be not be used for timer check. The old timer value will be used for check access.
Parameters (out):	None	
Return value:	None	
Description:	Executes check access to verify password/timer (optional) available for CPU0, CPU1 and CPU2 watchdogs Given password and timer are used on check access	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

5.2.37 Mcal_SafetyPwSequence

Table 43 Specification of API Mcal_SafetyPwSequence

Syntax:	uint32 Mcal_SafetyPwSequence (const uint32 Password)	
Parameters (in):	Password	start password to be used for calculation

		of resulting password value
Parameters (out):	None	
Return value:	Uin32	
Description:	Calculates the expected next password and returns the valid password according to password mode for the Safety Watchdog.	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

5.2.38 Mcal_CpuWdtPwSequence

Table 44 Specification of API Mcal_CpuWdtPwSequence

Syntax:	Uin32 Mcal_CpuWdtPwSequence (const uin32 Password)	
Parameters (in):	Password	start password to be used for calculation of resulting password value
Parameters (out):	None	
Return value:	Uin32	
Description:	Calculates the expected next password and returns the valid password according to password mode for the CPU Watchdogs.	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

5.2.39 Mcal_SuspendAllInterrupts

Table 45 Specification of API Mcal_SuspendAllInterrupts

Syntax:	inline void Mcal_SuspendAllInterrupts(void)	
Parameters (in):	None	
Parameters (out):	None	
Return value:	None	
Description:	This service disables all the enabled interrupts. On error Mcal_SafeErrorHandler is called. Please refer description of Mcal_SafeErrorHandler for details.	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

5.2.40 Mcal_ResumeAllInterrupts

Table 46 Specification of API Mcal_ResumeAllInterrupts

Syntax:	inline void Mcal_ResumeAllInterrupts(void)	
Parameters (in):	None	
Parameters (out):	None	
Return value:	None	
Description:	This service restores all the disabled interruptsOn error Mcal_SafeErrorHandler is called. Please refer description of Mcal_SafeErrorHandler for details.	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

5.2.41 Mcal_GetCoreId

Table 47 Specification of API Mcal_GetCoreId

Syntax:	uint8 Mcal_GetCoreId
----------------	----------------------

Parameters (in):	None	
Parameters (out):	None	
Return value:	Core ID - The identification number of the core	
Description:	Function to get the identification number of the core.	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

5.2.42 Mcal_GetDsprReMapAddress

Table 48 Specification of API Mcal_GetDsprReMapAddress

Syntax:	uint32 Mcal_GetDsprReMapAddress(uint32 Address)
Parameters (in):	Address
Parameters (out):	None
Return value:	Data scratch pad SRAM register Map address
Description:	Function returns Data scratch pad SRAM register map address
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.

5.2.43 Mcal_GetCpuAddress

Table 49 Specification of API Mcal_GetCpuAddress

Syntax:	lfx_CPU* Mcal_GetCpuAddress (uint8 Cpuld)
Parameters (in):	Cpuld
Parameters (out):	None
Return value:	lfx_CPU*
Description:	This function get the CPU base address of the provided CPU ID
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.

5.2.44 Mcal_SetCpuPC

Table 50 Specification of API Mcal_SetCpuPC

Syntax:	void Mcal_SetCpuPC (uint8 CpuNo, uint32 ProgramCounter)	
Parameters (in):	CpuNo	CPU index
	ProgramCounter	Address to be updated in PC register
Parameters (out):	None	
Return value:	None	
Description:	This function updates the PC register of the corresponding CPU with the provided address	
I/O Mode:	This API should be invoked only when the CPU is in Supervisor Mode.	

5.2.45 Mcal_StartCore

Table 51 Specification of API Mcal_StartCore

Syntax:	void Mcal_StartCore (uint8 CpuNo, uint32 Pcval)	
Parameters (in):	CpuNo	CPU index
	Pcval	Address to be updated in PC register

Parameters (out):	None
Return value:	None
Description:	This function updates the PC register of the corresponding CPU with the provided address and puts the CPU to run mode
I/O Mode:	This API should be invoked only when the CPU is in Supervisor Mode.

5.2.46 Mcal_GetSpinLock

Table 52 Specification of API Mcal_GetSpinLock

Syntax:	void Mcal_GetSpinLock (uint32* SpinLckPtr,uint32 Timeout)	
Parameters (in):	SpinLckPtr	Spinlock to be acquired
	Timeout	Wait duration for acquisition of SpinLock
Parameters (out):	None	
Return value:	None	
Description:	This function waits in a loop till it successfully acquires the spinlock or max no of times given by TimeOut.	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

5.2.47 Mcal_ReleaseSpinLock

Table 53 Specification of API Mcal_ReleaseSpinLock

Syntax:	void Mcal_ReleaseSpinLock (uint32* SpinLckPtr)	
Parameters (in):	SpinLckPtr	Spinlock to be released
Parameters (out):	None	
Return value:	None	
Description:	This function releases the acquired spinlock.	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

5.2.48 Mcal_SafeErrorHandler

Table 54 Specification of API Mcal_SafeErrorHandler

Syntax:	void Mcal_SafeErrorHandler(uint32 ErrorType)	
Parameters (in):	ErrorType	Type of Error occurred. 1. MCAL_DIV_INCONSISTENCY 2. MCAL_SPINLOCK_TIMEOUT
Parameters (out):	None	
Return value:	None	
Description:	Mcal_SafeErrorHandler function is called in two conditions - Any detection of inconsistency in copies of relevant global variables is reported to the user via the function Mcal_SafeErrorHandler. Mcal_SafeErrorHandler shall never return, else it may lead to unpredictable behavior. The user is expected to perform the error handling in this function. The error reported is MCAL_DIV_INCONSISTENCY: Typical error reactions could be to reset the microcontroller (software reset) - Support functions are called to access the protected registers, these functions can be called from multiple cores concurrently, and a spinlock	

	mechanism with timeout is provided. In case, the timeout expires, it is reported to the user via Mcal_SafeErrorHandler. The user is expected to perform the error handling in this function. The error reported is MCAL_SPINLOCK_TIMEOUT: Typical error reactions could be to terminate the operating system task or restart the partition or reset the microcontroller. The user needs to implement this functionality (Mcal_SafeErrorHandler) in a separate file and needs to add this file as part of the build.
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.

5.2.49 Mcal_LockResource

Table 55 Specification of API Mcal_LockResource

Syntax:	uint32 Mcal_LockResource(uint32 *ResourcePtr)	
Parameters (in):	ResourcePtr	Pointer to variable representing resource
Parameters (out):	None	
Return value:	uint32	
Description:	This function is an implementation of a binary semaphore using the "CmpAndswap" instruction of tricore. Allowed values are 0 and 1 for this binary semaphore. If a resource is free, the semaphore is taken and status MCAL_RESOURCE_FREE is returned. If the resource was already taken, a busy status MCAL_RESOURCE_BUSY is returned.	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

5.2.50 Mcal_UnlockResource

Table 56 Specification of API Mcal_UnlockResource

Syntax:	void Mcal_UnlockResource(uint32* ResourcePtr)	
Parameters (in):	ResourcePtr	Pointer to variable representing resource
Parameters (out):	None	
Return value:	None	
Description:	This function Frees the Resource already locked. Sets resource atomically to MCAL_RESOURCE_FREE = 0.	
I/O Mode:	This API can be invoked when the CPU is in either User-1 Mode or Supervisor Mode.	

5.3 Access Permissions

The below mentioned functions can be called directly from QM and ASIL SW (same MPU access protection rights must be granted for QM and ASIL software, in case the provided APIs are being called from QM and ASIL SW):

Function Name	File
Mcal_ResetENDINIT	Mcal_WdgLib.c
Mcal_SetENDINIT	Mcal_WdgLib.c
Mcal_CpuWdtModifyAccess	Mcal_WdgLib.c

Mcal_SafetyPwSequence	Mcal_WdgLib.c
Mcal_CpuWdtPwSequence	Mcal_WdgLib.c
Mcal_ResetSafetyENDINIT_Timed	Mcal_WdgLib.c
Mcal_SetSafetyENDINIT_Timed	Mcal_WdgLib.c
Mcal_ResumeAllInterrupts	Mcal_TcLib.c
Mcal_SuspendAllInterrupts	Mcal_TcLib.c
Mcal_ReleaseSpinLock	Mcal_TcLib.c
Mcal_GetSpinLock	Mcal_TcLib.c
Mcal_UnlockResource	Mcal_TcLib.c
Mcal_LockResource	Mcal_TcLib.c

In case the provided APIs are being called from QM and ASIL SW, all variables used in the two files shall be allocated to the same MPU context which allows accessing them from QM and ASIL SW.

The following failure modes have been considered for the same:

Failure Mode	Counter Measure
Overwrite of Global variables from corrupted SW	Double inverse storage of variables
Overwrite of the register value (like WDT_CONx) from corrupted SW	HW measure (call sequence) in place
Unexpected execution of these function from corrupted SW: i. Register protection (EndInit, Safe EndInit) wrongly switched off: ii. Register protection (EndInit, Safe EndInit) Not switched on:	HW Measure, Reset the EndInit bit puts the watchdog into the Time-Out mode. If the ENDINIT is not SET within a defined time, it results in an SMU alarm

6 Example Usage

This chapter describes how the MCAL library APIs can be used in different drivers

Configuration of the DMA is done in MCU driver.

Code Listing 1 Example – MCAL LIB usage in ADC driver

```

001:    Dma_ChannelType          DmaChannel;
002:
003:    KernelIndex = Adc_kKernelIndex[AdcKernel];
004:
005:    DmaChannel = Adc_kKernelCfgPtr[AdcKernel]->HwCfgPtr->DmaChannel;
006:    /*-----**
007:    ** Interrupt is because of ADC0 Group EMUX SCAN **
008:    **-----*/
009:    /* Clear interrupt on DMA Channel */
010:    Mcal_DmaChClrIntrFlags(DmaChannel);
011:
012:    /* Disable DMA Channel on Hardware request */
013:    Mcal_DmaDisableHwTransfer(DmaChannel);
014:
015:    /* Get the ADC0:Group Id from Adc_DmaData */

```

Code Listing 1 Example – MCAL LIB usage in ADC driver

```
016:      GrpNr = Adc_DmaData.Group;
```

Code Listing 2 Example – MCAL LIB usage in SPI driver

```
017:      #endif /* (SPI_LEVEL_DELIVERED == 2U) */
018:      {
019:          /*QSPiX Bus used in Async Mode in Level 2*/
020:          #if ((SPI_QSPI0_USED == STD_ON) && (SPI_QSPI0_HWTYPE == ASYNC_BUS))
021:              Module = SPI_QSPI0_INDEX;
022:              DmaTransmitChannel = Spi_DmaChannelIdx[Module].TxDmaChannel;
023:              DmaReceiveChannel = Spi_DmaChannelIdx[Module].RxDmaChannel;
024:              DmaErrorstatus = Mcal_DmaGetTransReqLostFlag(DmaTransmitChannel);
025:              DmaErrorstatus |= Mcal_DmaGetMoveEngErrFlags(DmaTransmitChannel);
026:              DmaErrorstatus |= Mcal_DmaGetTransReqLostFlag(DmaReceiveChannel);
027:              DmaErrorstatus |= Mcal_DmaGetMoveEngErrFlags(DmaReceiveChannel);
028:
029:              Spi_lMainFunctionErrorCheck(DmaErrorstatus,Module,\
030:              DmaTransmitChannel,DmaReceiveChannel);
031:          #endif /* (SPI_QSPI0_USED == STD_ON) */
```

Figure 4 Mcal_UpdateRegSV execution when IFX_MCAL_RUN_MODE_DEFINE = 0U (Supervisor Mode)

Figure 5 Mcal_UpdateRegSV execution when IFX_MCAL_RUN_MODE_DEFINE = 1U (User-1 Mode without OS)

Figure 6 Mcal_UpdateRegSV execution when IFX_MCAL_RUN_MODE_DEFINE = 2U (User-1 Mode with OS)

7 Assumptions and Limitations

None

www.infineon.com